

Chapter 1

Squeak Peek Studio Documentation

Contact

Antonín Gazda

Prague, Czech Republic

Current App Version v2.0

Latest Update 29. 12. 2025

me@antoningazda.com

antoningazda.com/squeakpeek

Below is a walkthrough of the main panels in the current Squeak Peek Studio application, presented in the order they appear in the user interface.

1.1 Application GUI

1.1.1 Data Input

The **Data Input** panel (Figs. 1.1 and 1.2) is where users load the essential input files: the raw USV audio files, manually annotated reference labels, and optionally, pre-computed detection labels. The application supports two modes of operation: *single-file mode* and *batch mode*.

In single-file mode, users work with a single audio recording and its corresponding labels. This mode is typically used for visualization, manual inspection, and editing of USVs. Batch mode, on the other hand, allows users to select multiple files and process them automatically. In this mode, the app displays the number of files loaded for each category and provides dropdown menus for selecting the active file for further interaction.

After selecting the paths, pressing the “Load Files” button imports the data into the app and prepares it for subsequent operations such as segmentation, sonification, or visualization.

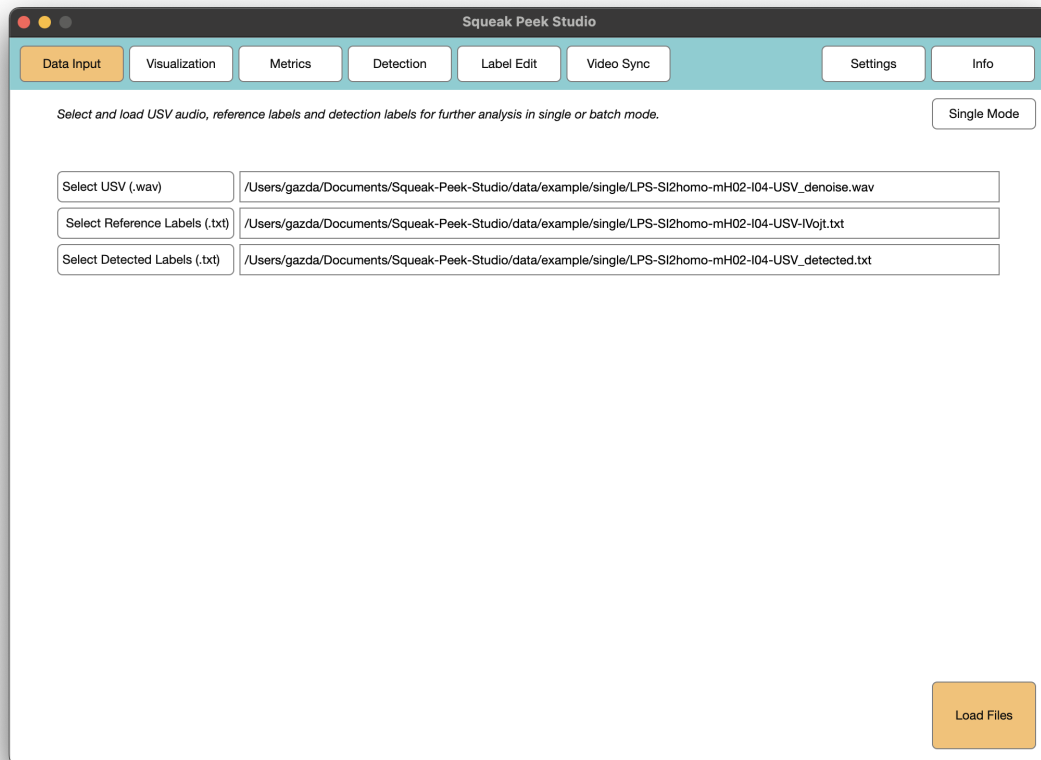


Figure 1.1: Data Input panel in single-file mode.

Squeak Peek Studio

Data Input
Visualization
Metrics
Detection
Label Edit
Video Sync
Settings
Info

Select and load USV audio, reference labels and detection labels for further analysis in single or batch mode.
Batch Mode

Select USVs (.wav)
/Users/gazda/Documents/Squeak-Peek-Studio/data/example/batch/USVs
(5)

Select Reference Labels (.txt)
/Users/gazda/Documents/Squeak-Peek-Studio/data/example/batch/referenceLabels
(5)

Select Detected Labels (.txt)
/Users/gazda/Documents/Squeak-Peek-Studio/data/example/batch/labels
(5)

You are in batch mode. To use single-file operations such as Visualization, please select the file to work with.

Select USV
LPS-SI2homo-mH02-I04-USV_denoise.wav

Select Reference Label
LPS-SI2homo-mH02-I04-USV-IVojt.txt

Select Detected Label
LPS-SI2homo-mH02-I04-USV_denoise_PSD.txt

Load Files

Figure 1.2: Data Input panel in batch mode.

1.1.2 Visualization

The **Visualization** panel (Fig. 1.3) displays the currently selected segment of the USV recording in both time-domain waveform and frequency-domain spectrogram. Users can specify the segment start time and length, navigate forwards/backwards through the recording, and click “Sonify” to play a human-audible version of the ultrasonic segment.

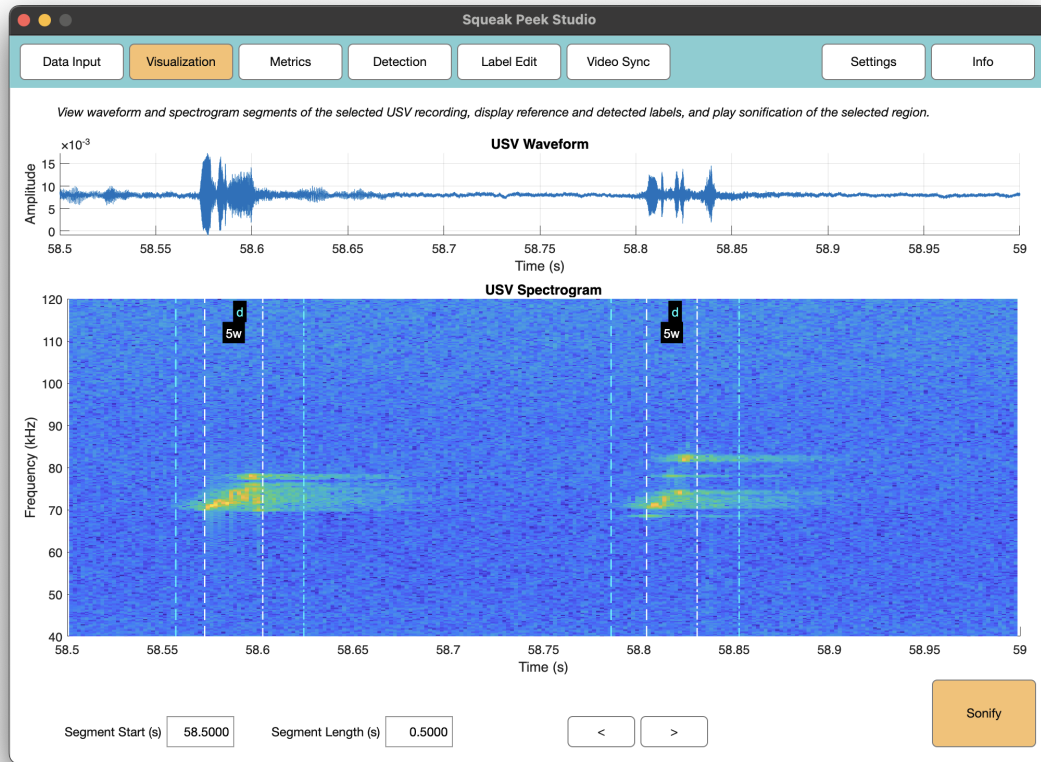


Figure 1.3: Visualization panel: waveform and spectrogram view, segment controls, navigation buttons, and sonification.

1.1.3 Metrics

In the **Metrics** panel (Fig. 1.4), the app computes standard detection performance measures by comparing detected labels against reference labels. It reports the total counts of detected and reference calls, true positives and false negatives, and derived precision, recall, and F1 score.

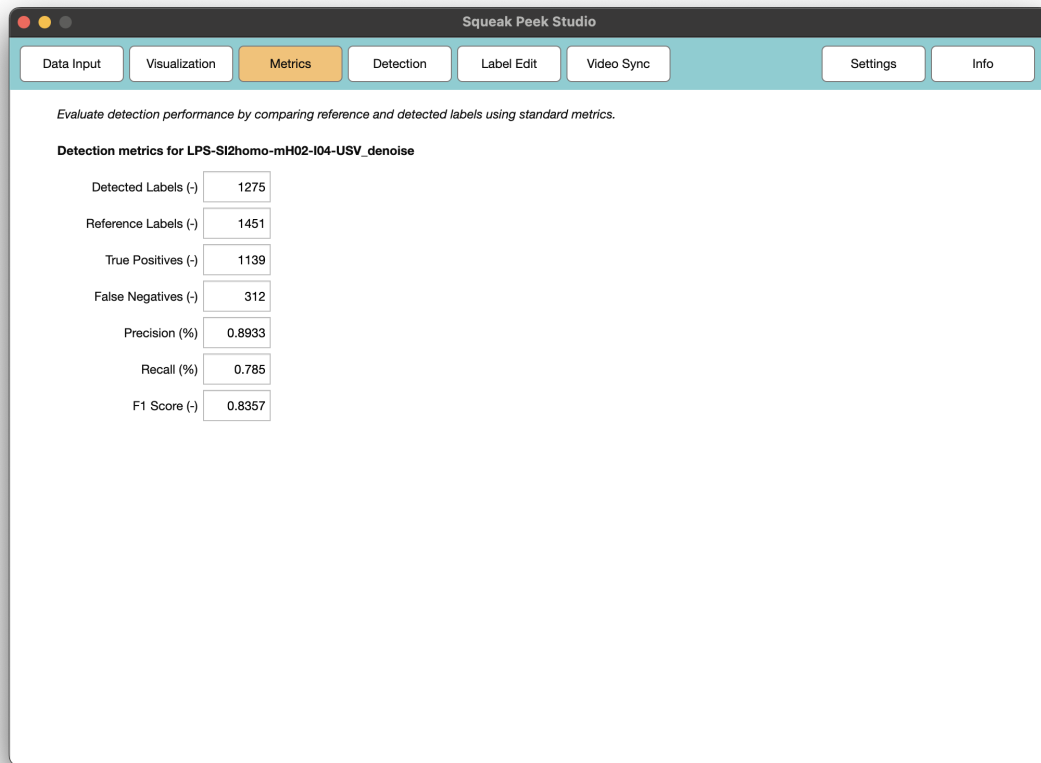


Figure 1.4: Metrics panel: detection counts and performance metrics (precision, recall, F1).

1.1.4 Detection

The **Detection** panel (Fig. 1.5) allows users to choose one or more detection algorithms (e.g. PSD, BSCD, RBD) and optional post-processing steps (merging close labels, removing short labels). After selecting an export folder, clicking “Run Detectors” will batch-process all loaded audio files and save newly generated label files.

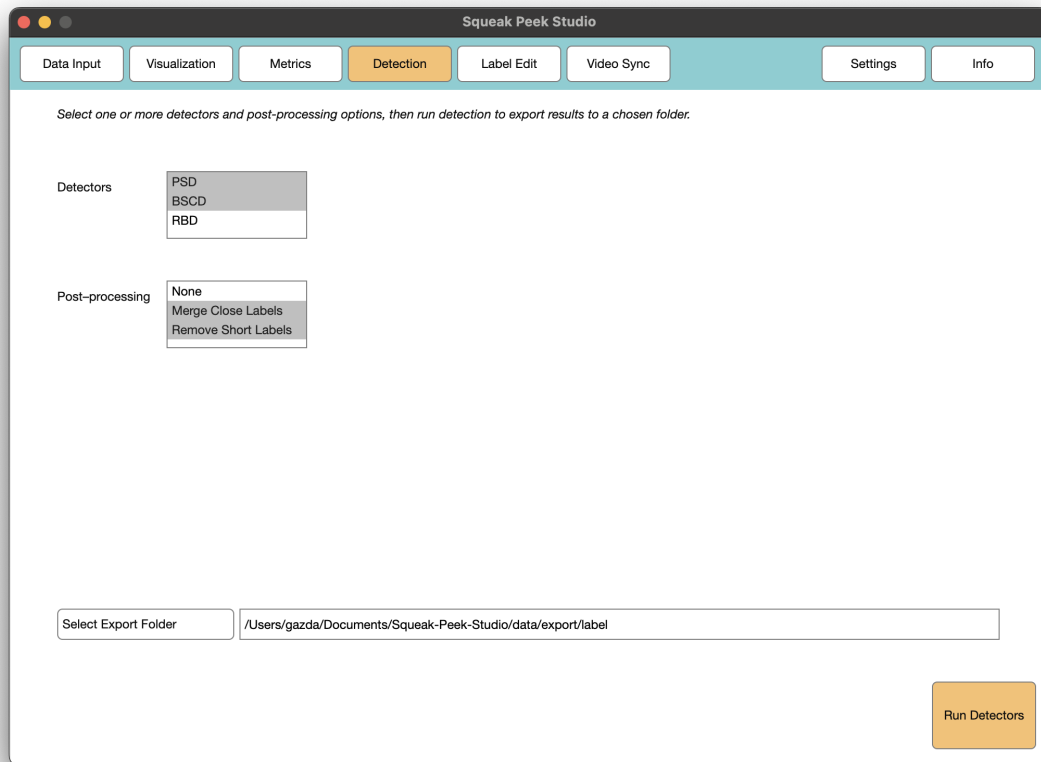


Figure 1.5: Detection panel: select detectors and post-processing, choose export folder, and run detection.

1.1.5 Label Editing

The **Label Edit** panel (Fig. 1.6) provides manual curation tools. Users can drag the call-boundary handles on the spectrogram to fine-tune start/end times, accept or reject each detected call and its classification label, and step through calls one by one. Counters for accepted/rejected detections and classifications are shown, and “Export” saves the edited labels.

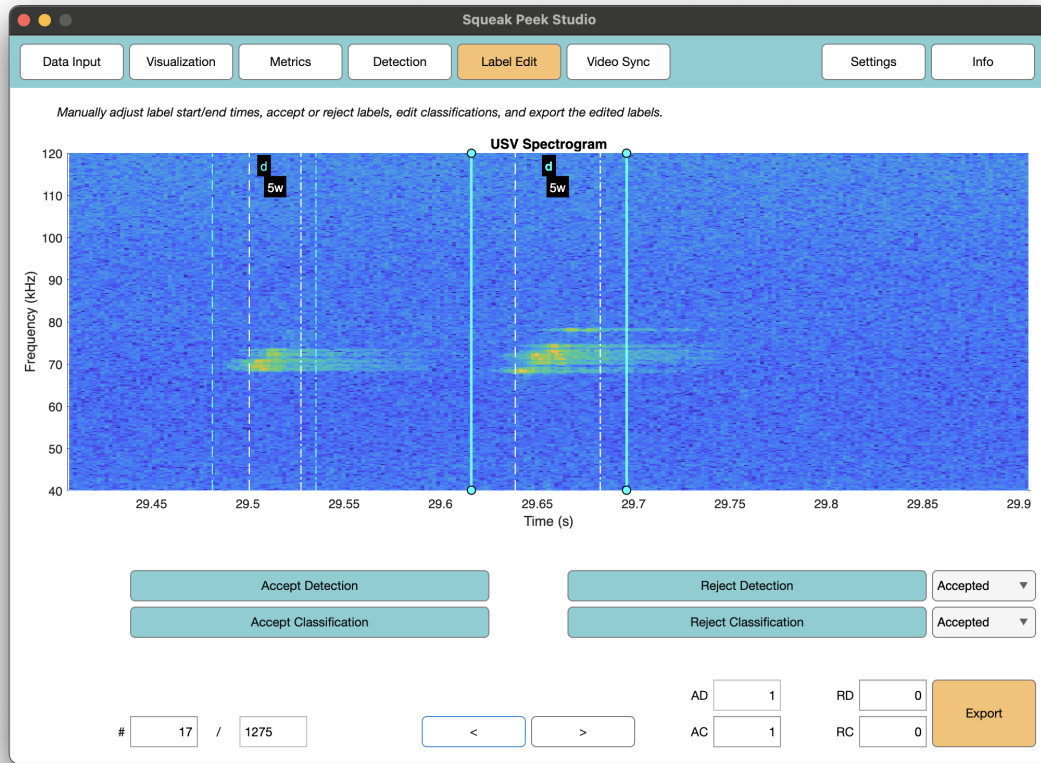


Figure 1.6: Label Edit panel: adjust boundaries, accept/reject detections and classifications, navigate through calls, and export.

1.1.6 Video Synchronization

The **Video Sync** panel (Fig. 1.7) synchronizes an external video recording with the USV audio. Users select the video file, enter the synchronization timestamp (when the first USV call occurred), choose whether to overlay the original or sonified audio, and click “Synchronize” to produce a trimmed, audio-replaced video.

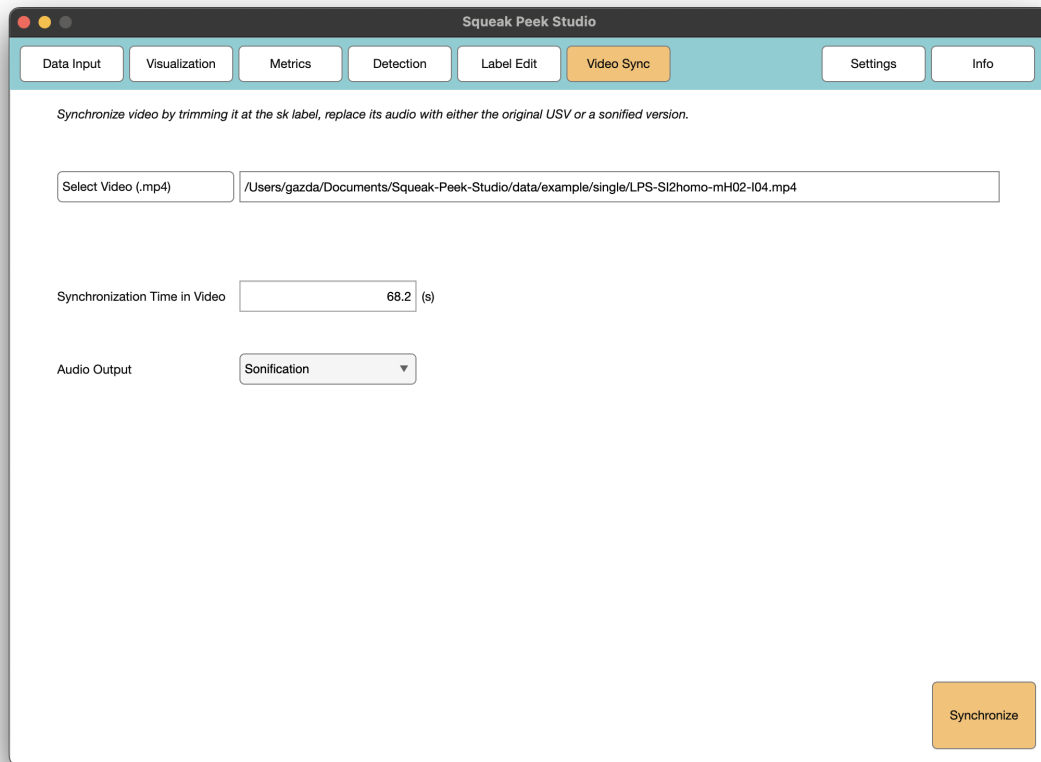


Figure 1.7: Video Sync panel: select video, set sync time, choose audio output, and synchronize.

1.1.7 Settings

The **Settings** panel provides centralized access to all configurable parameters within the application. It is context-sensitive—when accessed from a specific module (e.g., Data Input), it automatically opens the corresponding tab, streamlining navigation and improving usability. All settings can be exported and imported as structured `.json` files, enabling reproducibility and easy sharing across users or sessions. These files can also be edited outside of MATLAB. On startup, the application automatically looks for a `default.json` file; if present, it loads this configuration, restoring the app to its exact previous state. This ensures that users return to the same working environment, facilitating continuity and personalized workflows.

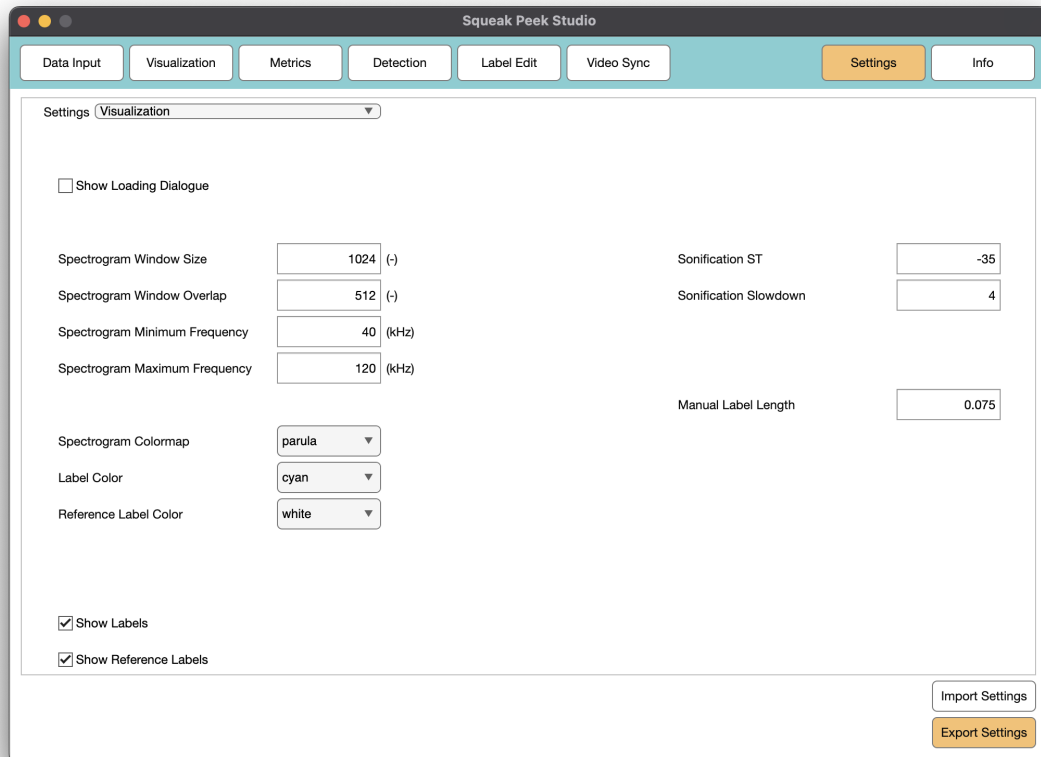


Figure 1.8: Settings panel: centralized configuration management and default profile handling.

1.1.8 Info

Finally, the **Info** panel (Fig. 1.9) displays application meta title, description, author and supervisor details, contact, academic year, MATLAB and app version numbers, and quick-link buttons to documentation, source code, and the full Master's thesis. Institutional logos are shown at the bottom.

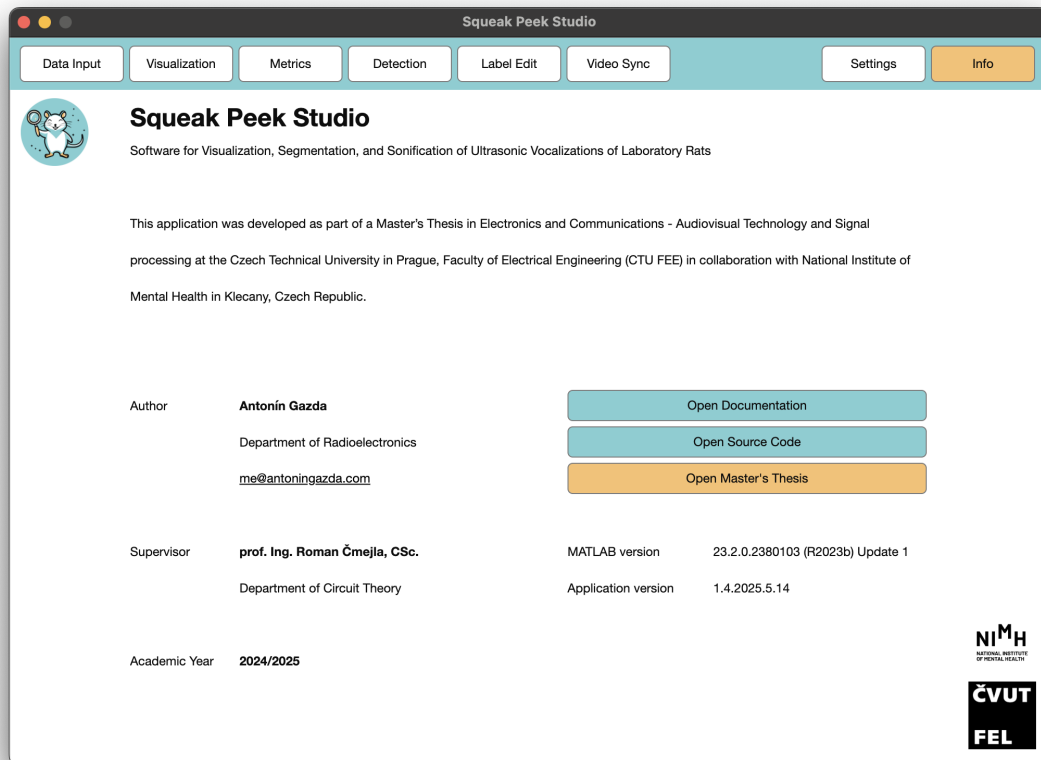


Figure 1.9: Info panel: application metadata, author/supervisor info, version numbers, and links.

1.2 Packaging

The *Squeak Peek Studio* application was packaged as a *MATLAB App*, allowing for straightforward distribution and installation within the environment. The application is provided as an `.mlappinstall` file, which can be installed by simply double-clicking it. Once installed, the app becomes accessible from the Apps gallery with a single click.

This packaging format is ideal for researchers already working within the ecosystem, as it ensures seamless integration, version control, and portability across different systems running MATLAB.

In addition to MATLAB App packaging, the application can also be compiled into a standalone desktop application using the *MATLAB Application Compiler*. This generates a platform-specific installer, which includes the MATLAB Runtime, enabling users to run the app on systems that do not have MATLAB installed.

Furthermore, with minimal modification, the application can be compiled and deployed as a web app using the *MATLAB Web App Server*. This allows the application to be accessed remotely through a web browser, providing a convenient interface for users who prefer not to install any software locally.

All packaging options are supported by MATLAB's built-in tools, making the app flexible to deploy and distribute depending on the user's environment and needs.

1.3 Installation and Updating

Squeak Peek Studio is distributed as a packaged MATLAB App Installer file (`.mlappinstall`) and can be easily installed or updated through the MATLAB interface.

Installation via `.mlappinstall`

To install the application:

1. Double-click the provided `Squeak Peek Studio.mlappinstall` file.
2. MATLAB will open a prompt asking whether you want to install the app (Figure 1.10).
3. Click **Install**. Once installed, the app appears under the **Apps** tab, ready to launch (Figure 1.11).

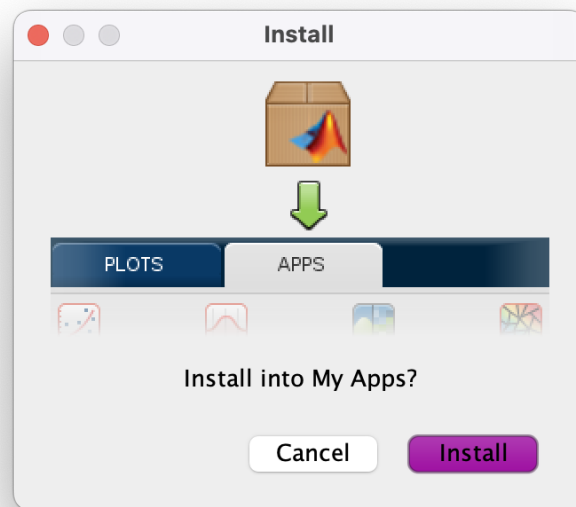


Figure 1.10: Installation prompt for the Squeak Peek Studio app.

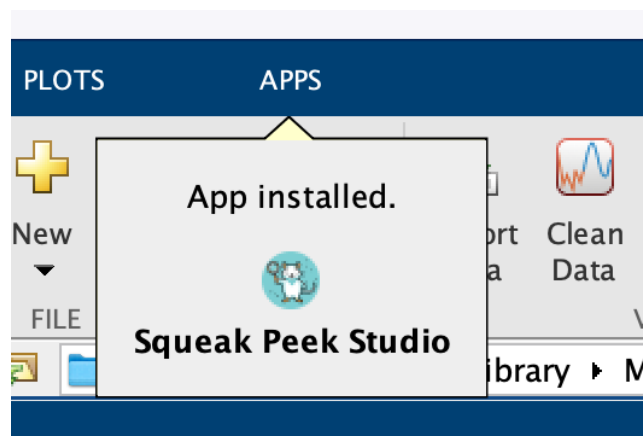


Figure 1.11: Squeak Peek Studio appears under the Apps tab after successful installation.

Updating the App

If an older version is already installed, MATLAB will prompt the user to confirm the update when a newer version is opened via `.mlappinstall` (Figure 1.12). Clicking **Update** replaces the older version while preserving its placement in the Apps tab.

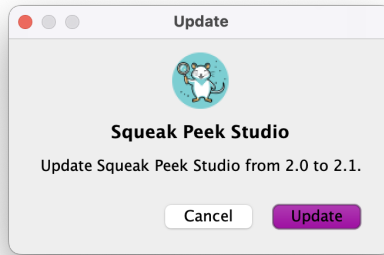


Figure 1.12: Update dialog for an existing installation of Squeak Peek Studio.

Development Mode via `runApp.m`

For users who wish to run the app in development mode (e.g., to access source code, debug features, or inspect runtime variables), the file `runApp.m` can be used. This script launches the application with all required paths initialized and retains access to the live session environment.

This method is recommended for developers or users interested in customizing detectors, modifying GUI elements, or extending functionality. Both `runApp.m` and the app package are listed in Appendix ??.

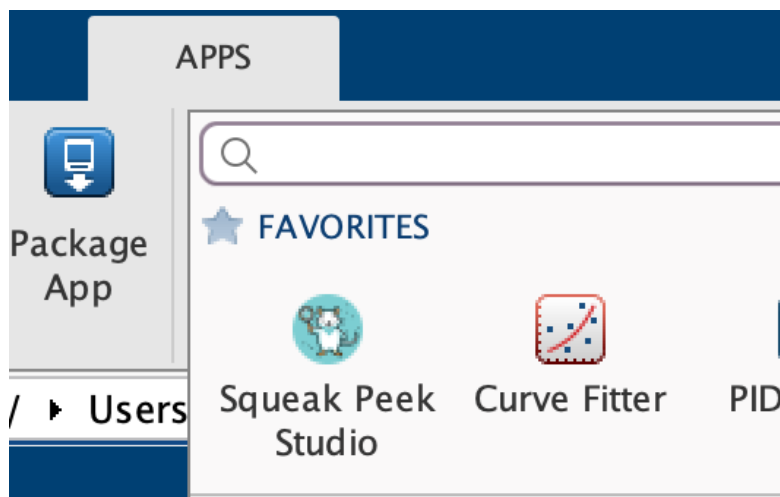


Figure 1.13: Squeak Peek Studio listed among other MATLAB Apps after installation.